

Introduction

Gradient Descent is one of the most widely used optimization algorithms in machine learning. It is an iterative method that helps minimize a cost function by adjusting model parameters step by step. At its core, gradient descent is about finding the lowest point of a curve (or surface in higher dimensions), which represents the best parameters for a given model.

How it Works

1. **Cost Function:** Represents the error between the predicted values and actual values. Example: Mean Squared Error in regression.
2. **Gradient:** The slope of the cost function with respect to model parameters.
3. **Update Rule:**

$$\theta := \theta - \alpha \cdot \nabla J(\theta)$$

θ : parameters of the model

α : learning rate

$\nabla J(\theta)$: gradient of the cost function

Types of Gradient Descent

- **Batch Gradient Descent:** Uses the entire dataset to compute gradients.
- **Stochastic Gradient Descent (SGD):** Updates parameters after each training example, faster but noisier.
- **Mini-Batch Gradient Descent:** Combines both, using small subsets of data.

Applications

- **Linear Regression:** Minimizing squared error to fit a line to data.
- **Logistic Regression:** Optimizing the likelihood function for classification.
- **Neural Networks:** Training deep learning models by minimizing complex cost functions.
- **Support Vector Machines:** Used in optimization of loss functions.

Challenges

- Choosing the right **learning rate**: too high → divergence, too low → slow training.
- **Local minima and saddle points**: especially in deep learning.
- Need for optimizers like **Adam, RMSProp, Momentum** to speed up convergence.